

# Exactness, Not Acyclicity: Which Homological Constraints Improve a Learned Conversion Between Structured Representations

Sean Mahdavian

Phillips Exeter Academy · ORCID [0009-0000-8432-7825](https://orcid.org/0009-0000-8432-7825)

Revision: 2026-08-24

## Abstract

---

Moving data between structured representations — graphs, cell complexes, sheaves — loses information, and homological algebra supplies exact tools for measuring what a map destroys. It is natural to hope those tools can also *improve* a learned conversion by acting as training objectives. We test three of them and find that the answer depends entirely on which object is chosen, and that the difference is predictable in advance from two cheap analytic checks.

Under a protocol frozen and committed before execution, on 29 topologies of a generator built so that conversion is learnable and homological defects vary, an **exactness** constraint improves a learned conversion by roughly two orders of magnitude in held-out error: Bonferroni-adjusted interval  $[-2.802, -1.458]$  on the paired  $\log_{10}$  ratio. The constraint is not leaked supervision — it is built from the graph the model already observes, while the answer is withheld. The resulting **exactness violation is a calibrated measure of damage**: within-topology correlation with held-out error of  $+0.854$ , 95% interval  $[+0.831, +0.877]$ , positive in 29 of 29 topologies.

The same campaign refutes the two objects this line of work more commonly reaches for. A **mapping-cone** term does not merely fail to help; it harms, adjusted interval  $[+0.102, +0.277]$ . A **representation-topology-divergence** term is inert,  $[-0.003, +0.039]$ .

We explain both failures mechanistically, and the explanations differ. In a second setting — selection among a fixed family of twelve maps on a cellular annulus — every candidate is a signed permutation, hence an isomorphism, so cone acyclicity is *constant* over the hypothesis class and carries zero information at any weight; models trained on it alone identify at chance while producing acyclic cones in 6,000 of 6,000 evaluated examples. In the learned-conversion setting the cone fails differently: the ground truth *violates* it, so the term pulls away from the answer. These two failure modes are exhaustive in our experience, and both are checkable in seconds. We give the check, and show it reproduces every outcome reported here without running a single fit.

We do not claim that homological structure teaches a model. We claim that one specific constraint, derivable from the input, measurably improves a learned conversion and calibrates its damage — and that the objects most often proposed for this role do not.

# 1. Introduction

---

## 1.1 The question

Given two structured representations of the same object, a conversion between them is a **typed map**: a family of linear maps, one per degree, required to commute with the boundary operators. Homological algebra measures what such a map destroys (the kernel of the induced map on homology), what it cannot reach (the cokernel), and whether it is an isomorphism (the acyclicity of its mapping cone).

The practical hope is that these measurements can serve as training signal — that attaching one as a loss term will make a learned conversion better. This paper asks which of them, if any, does.

## 1.2 What is and is not new

Learning higher-order structure from graphs is established: Differentiable Cell Complex Module [5] learns cell probabilities jointly with a task, Differentiable Lifting [6] learns liftings into cellular, simplicial, and combinatorial complexes, and Neural Sheaf Diffusion [7] and Knowledge Sheaves [8] learn sheaf restriction maps. Comparing paired representations through cross-barcode is likewise established: Representation Topology Divergence [1], its differentiable autoencoder form [2], and the symmetric variant SRTD [3] are the direct basis for our divergence reference, and a quotient-homology account of neural representation [4] develops an adjacent algebraic view. Categorical deep learning [9] and functor learning by gradient descent [10] ask what algebraic structure a learned map should respect.

Our contribution is narrow and largely negative-with-a-mechanism:

1. A confirmatory result that an **input-derivable exactness constraint** improves a learned conversion under scarce paired data, with a matched-rank control ruling out the obvious deflation.
2. A confirmatory result that the **exactness violation predicts damage**.
3. Confirmatory refutations of the **mapping cone** (harmful) and **RTD** (inert) as training terms, each with a distinct, stated mechanism.
4. A **screening criterion** — two analytic checks — that predicts all of the above without experiments, and a benchmark generator in which homological defects genuinely vary.

We make no claim of priority over any cited system, and §3 is positioning rather than systematic review.

# 2. Background and definitions

---

Each concept is stated in plain language before notation.

## 2.1 Graphs, cell complexes, and cellular sheaves

A **graph** records which pairs of objects are related. A **cell complex** adds higher-dimensional pieces: on top of vertices (degree 0) and edges (degree 1) it attaches faces (degree 2) glued along closed edge loops. A **cellular sheaf** attaches a small vector space to each cell and a linear transport map to each incidence, so data can move along the complex.

Formally a two-dimensional complex has boundary operators  $d_1$  (edges to vertices) and  $d_2$  (faces to edges) satisfying  $d_2 \circ d_1 = 0$ .

## 2.2 Typed maps, chain maps, exactness

A **typed map** carries degree- $n$  data to degree- $n$  data. It is a **chain map** when it commutes with the boundaries — informally, when it does not tear the complex. For a source with boundary  $d_C$  and target with boundary  $d_D$ ,

$$d_D \circ F_1 - F_0 \circ d_C = 0.$$

The left-hand side is the **exactness defect**. Our `ExactChainMapLayer` parameterizes  $(F_0, F_1)$  inside the nullspace of that expression, so exactness is architectural rather than penalized.

## 2.3 The implied complex

This paper's central move. When a model learns a lift  $W$  from edge signals to face signals,  $W^\top$  is a candidate face boundary. The learned conversion therefore **implies a complex** with boundaries  $(B_1, W^\top)$ , and that complex is legitimate only if  $B_1 \circ W^\top = 0$ . Every structural term below is written as a condition on the implied complex rather than bolted on:

| term  | form                              | meaning                                              |
|-------|-----------------------------------|------------------------------------------------------|
| exact | $\ B_1 \circ W^\top\ ^2$          | the implied complex satisfies $d \circ d = 0$        |
| cone  | $\exp(-2 \cdot \sigma_{\min}(W))$ | no implied 2-cell collapses                          |
| rtd   | distance-preservation proxy       | edge-signal geometry survives into face coefficients |

## 2.4 Kernels, cokernels, and mapping cones

The **kernel** of the induced map on homology is what a conversion destroys; the **cokernel** is what it cannot reach. A **mapping cone** is acyclic exactly when the map is a quasi-isomorphism, and its homology is the sum of the two:

$$\dim H_n(\text{cone } F) = \dim \text{coker } H_n(F) + \dim \ker H_{n-1}(F).$$

We verify this identity numerically on all 24 candidates of §4.1. It matters here because it shows the cone **bundles** two directional facts into one number and loses their separation — the kernel and cokernel are strictly more informative than the acyclicity bit.

## 2.5 RTD and SRTD

**Representation Topology Divergence** compares two point clouds describing the same items by tracking when topological features appear and disappear as a distance threshold grows. **SRTD** is a symmetric variant. Our corrected reference normalizes each dissimilarity matrix by its full-matrix 0.9 quantile, reports persistence separately by degree, uses degree 1 for the published scalar, and caps exact enumeration at 64 entities.

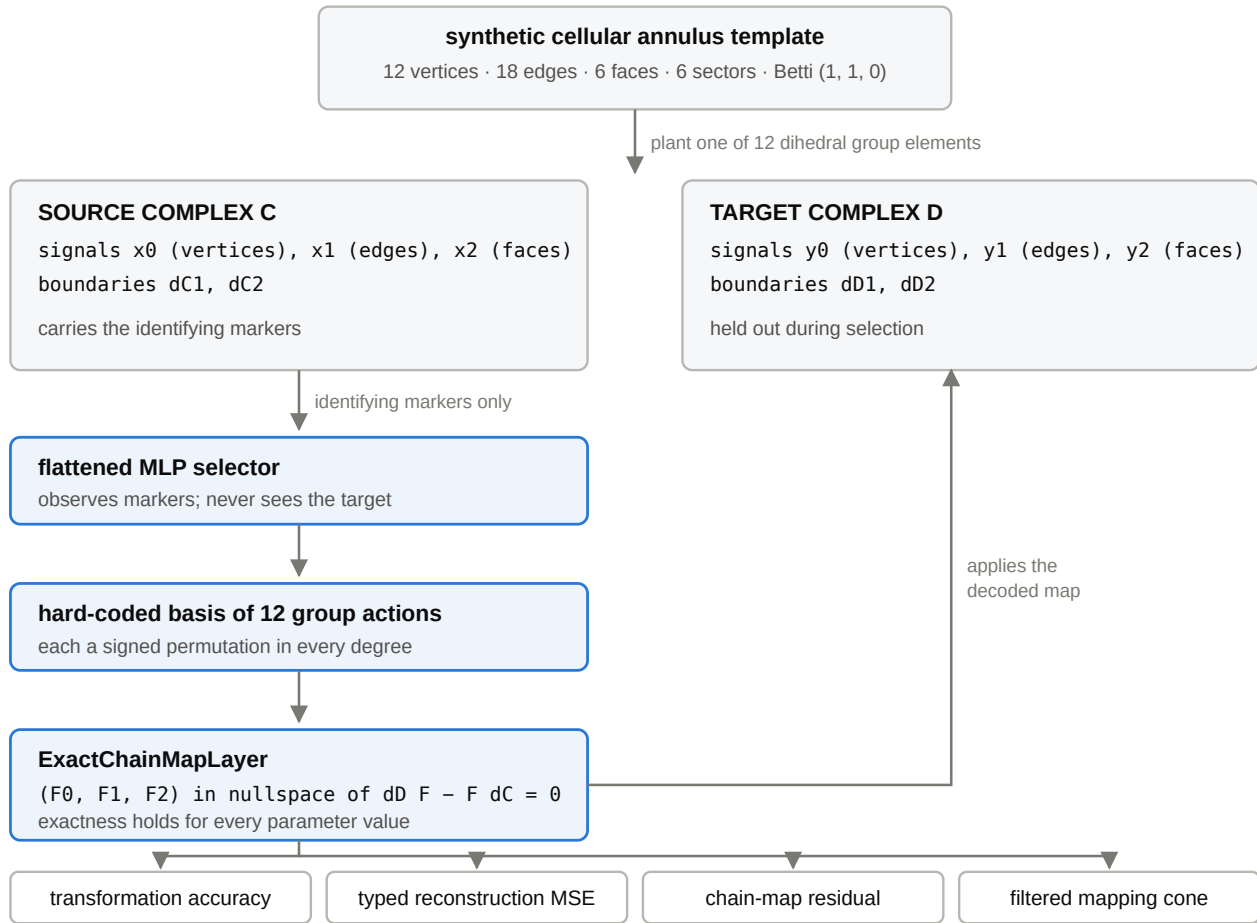
## 3. Related work

§1.2 and §2.5 cite the primary sources. In summary: RTD [1], RTD-AE [2], and SRTD [3] define the divergence reference, with quotient homology [4] an adjacent algebraic treatment; DCM [5] and DiffLift [6] are the closest precedents for learning higher-order topological structure from graphs; Neural Sheaf Diffusion [7] and Knowledge Sheaves [8] for learned sheaf transports; DeepMoE [11], GMoE [12], GraphMETRO [13], and MvCGE [14] for per-example expert routing.

Categorical deep learning [9] argues architectures are usefully described as algebras over a theory, and functor learning [10] gives an instance where the learned object must preserve structure rather than merely fit data. Our nullspace-constrained map sits in that tradition operationally; §10 states plainly that nothing here validates a categorical claim.

## 4. Two settings

### 4.1 Selection over a fixed family



**Figure 1. The selection setting.** A group element is planted in a six-sector cellular annulus (12 vertices, 18 edges, 6 faces, Betti (1, 1, 0)). The selector observes only identifying markers and never sees the target complex. It chooses one member of a fixed twelve-element basis, and the resulting typed map is scored four ways. The chain-map constraint is structural — it holds for every parameter value — so the residual is a numerical check, not a learned objective.

Twelve dihedral maps act on the annulus. Each is built as a **signed permutation in every degree**:  $F_0$  permutes vertices,  $F_1$  permutes edges with an orientation sign,  $F_2$  is fixed by matching the mapped boundary to a unique oriented face. We verify numerically that all twelve satisfy  $F^\top F = I$  at degrees 0, 1, and 2.

That property is the whole story of this setting, and §5 draws it out.

## 4.2 Learned continuous conversion

A generator in which conversion is genuinely learnable. Its design is forced by a conflict: the routing benchmark used elsewhere in this project deliberately hides cell structure from the graph observation so a router cannot shortcut, which makes conversion impossible by construction. Routing needs targets *not* inferable from the graph; conversion needs them *inferable*. One generator cannot serve both, so this is a separate one.

- The **2-cells are a cycle basis of the graph**, so the cell complex is determined by the graph rather than drawn beside it. Graph size and density vary, so the cycle rank varies, and the defect of a conversion varies with it.
- **Face activity thresholds the circulation** of the edge cochain around each cycle — exactly  $B_2^\top \times 1$ . Integrating an edge feature around a cycle is the operation a converter must perform.
- **Sheaf transport** combines an endpoint frame difference with a per-edge twist. The frame difference telescopes to the identity around any closed cycle, so without the twist every holonomy would be trivially zero; the twist rides a separate edge channel so holonomy and face activity are not the same quantity twice.

Over 300 samples the construction gives cycle ranks 0–31, **32 distinct defect profiles**, and  $B_1 B_2 = 0$  exactly. The degree-1 kernel of the canonical inclusion equals the cycle rank: it counts the cycles the conversion destroys.

**The learning task.** Learn  $W: R^E \rightarrow R^F$  with the cycle basis  $B_2$  **withheld**;  $B_1$  is observable because it is the graph. Ground truth is  $W = B_2^\top$ , a median of 216 free parameters.

## 5. When can a structural term carry information?

---

Two conditions are necessary, and in our experience jointly sufficient to avoid wasting a campaign.

**Condition 1 — the ground truth must satisfy the term.** Otherwise the term pulls away from the answer and can only hurt.

**Condition 2 — the term must vary over the hypothesis class.** A term that takes the same value on every reachable candidate carries zero information, at any weight.

Both conditions failed at least once in the experiments below, and the failures are instructive because they are different.

### 5.1 Condition 2 fails on the annulus, provably

A mapping cone is acyclic exactly when its chain map is a quasi-isomorphism. Every candidate in §4.1 is a signed permutation, hence invertible, hence a quasi-isomorphism. **Cone acyclicity therefore takes the same value on all twelve hypotheses.**

The same argument covers RTD there. A signed permutation is orthogonal, hence an isometry, so the mapped point cloud carries the same pairwise dissimilarity matrix as the source under every candidate — maximum observed distance change  $1.5e-07$  at float precision. The paired matrices RTD consumes are identical across the class.

The generalization is broader than this annulus: **any hypothesis class whose candidates are all invertible has a constant cone-acyclicity signal**, and that is precisely the class a practitioner has in mind when a cone objective looks attractive.

The obvious objection — that this is circular, because the class was chosen to consist of isomorphisms — restates the finding rather than rebutting it. Wanting a learned map to be structure-preserving is the same property that makes acyclicity useless for choosing *among* structure-preserving candidates. What makes it a trap rather than a triviality is the view from inside the training loop: the objective is driven to full satisfaction, every example returns a clean certificate, and identification never rises above chance.

### 5.2 Condition 1 fails on the learned conversion

In §4.2 the truth  $W = B2^\top$  is rank-deficient in the relevant sense, so a term rewarding large singular values is *violated by the answer*. A cone-flavoured penalty there does not fail to help; it actively pushes away from the truth. §7.3 confirms this.

### 5.3 The check is cheap

Both conditions are computable without training. `screen_structural_term` evaluates a candidate term on the ground truth and on a sample of the hypothesis class, and reports one of three verdicts: `satisfied-and-varies`, `ground-truth-violates-the-term`, or `constant-over-the-hypothesis-class`.

Applied retroactively it reproduces every outcome in this paper: exactness on the conversion task is usable; the cone on the conversion task is violated by the truth; the cone on the annulus is constant. **No fits are required.** We recommend running it before freezing any protocol around a new structural term.

## 6. Experimental design

### 6.1 Preregistration

The conversion campaign was frozen in a protocol document committed **before execution**: SHA-256 503cc282f40d118ba1739c2afe1bfc77eaf2b1733baaddb91c0c3363e75ae2b8, committed at d5d18af, campaign run at 11644c6 from a clean worktree. No endpoint, weight, decision rule, or sample size changed after the freeze.

| item                | value                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------|
| declared topologies | 30 (seeds 20261001–20261030)                                                              |
| eligible            | <b>29</b> ; seed 20261025 had fewer than three faces and was skipped, <b>not replaced</b> |
| training pairs      | 16                                                                                        |
| held-out pairs      | 3072                                                                                      |
| observation noise   | 0.02                                                                                      |
| optimiser           | Adam, lr 0.05, 2500 steps, <code>W</code> initialised to zeros, float64                   |
| weights             | exact 3.0, cone 0.01, rtd 0.1                                                             |

Each weight is the best-performing value for that term in prior exploratory work, so **every term is represented at its most favourable tested setting**. Weights are not comparable across terms because the terms have different scales.

### 6.2 Endpoints and decisions

Primary endpoint is the paired quantity, one value per topology:

d = log10( held-out MSE with term / held-out MSE without )

Negative means the term improves the model. Pairing removes topology variance.

**Multiplicity is adjusted.** The three primary contrasts form one family; the confirmatory interval is Bonferroni-corrected to two-sided 98.333%, giving a family-wise error rate of 0.05. Unadjusted 95% intervals are reported alongside and the adjusted interval governs. An exact two-sided sign test is reported as distribution-free sensitivity and governs nothing.

The campaign ran once over all declared topologies. No interim analysis, no early stop, no extension.

### 6.3 Hardware, software, provenance

Campaign: CPU, float64, PyTorch 2.13.0+cu130, Python 3.12.3. The annulus campaign and all timing benchmarks ran on one NVIDIA GB10, Linux 6.17.0-1026-nvidia aarch64, CUDA 13.0, deterministic algorithms enabled with CUBLAS\_WORKSPACE\_CONFIG=:4096:8.

| item                           | value                                                            |
|--------------------------------|------------------------------------------------------------------|
| annulus campaign commit        | 8021292e97abfec91768f1b5437c883a42c29c60                         |
| annulus launch fingerprint     | 44408d7adf8467e594879b46e25a1cb7fd89a7e7a5d5f3446548bcbf3ed1096e |
| annulus strict summary SHA-256 | 0cd0defb0b0d41b5f7563c364cfdda62cb72c5e5a845bdd5d1ab76a2e1cb953c |
| routing campaign commit        | e69b07707950b6abe332366c51fe8c94254899f3                         |
| scheduler steps completed      | 56 of 56                                                         |

The sealed scheduler receipt lists all 296 produced files with byte counts and SHA-256 digests; all 296 verify against the on-disk artifacts.

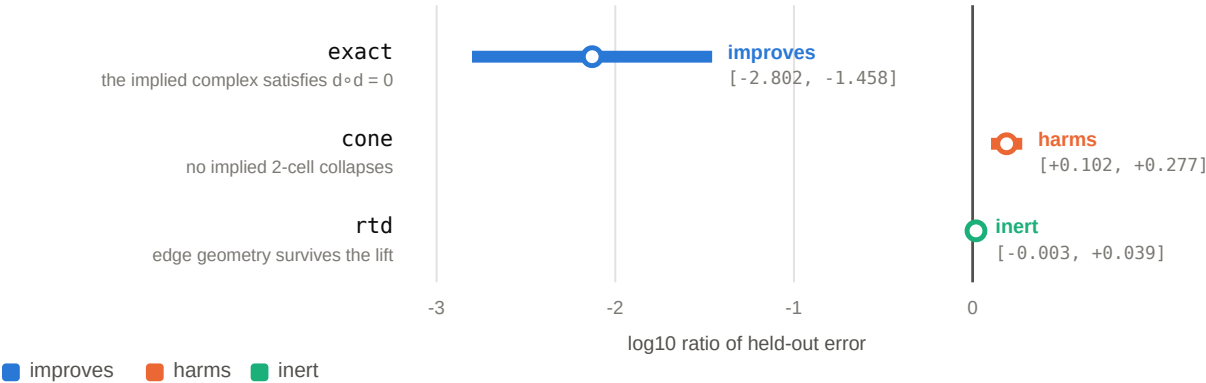
## 7. Results

### 7.1 Exactness improves a learned conversion

| term  | weight | 95% interval     | Bonferroni 98.33% | sign test | decision |
|-------|--------|------------------|-------------------|-----------|----------|
| exact | 3.0    | [−2.628, −1.632] | [−2.802, −1.458]  | <1e-6     | improves |
| cone  | 0.01   | [+0.125, +0.254] | [+0.102, +0.277]  | <1e-6     | harms    |
| rtd   | 0.1    | [+0.002, +0.034] | [−0.003, +0.039]  | 0.458     | inert    |

Which homological term improves a learned conversion?

Paired  $\log_{10}(\text{held-out error with term} / \text{without})$ , one value per topology. Negative means the term helps.  
29 topologies, 16 training pairs, preregistered in docs/27.  
Thick bar: Bonferroni 98.33% (governs the decision). Thin bar: unadjusted 95%.



**Figure 5. The three homological terms, one protocol, three answers.** Paired  $\log_{10}(\text{held-out error with term} / \text{without})$ , one value per topology, 29 topologies. Thick bar is the Bonferroni-adjusted 98.33% interval that governs the decision; thin bar is the unadjusted 95%. Exactness improves by roughly two orders of magnitude; the cone harms; RTD is inert. Generated from `results/campaigns/conversion-campaign-v1.json`.

Median  $\log_{10}$  ratio for exactness is  $-2.858$ , roughly a **700-fold** reduction in held-out error; the mean is  $-2.130$ . The adjusted interval lies far from zero.

The term is  $\|W B_1^\top\|^2$ , satisfied exactly by the truth because  $B_2^\top B_1^\top = (B_1 B_2)^\top = \mathbf{0}$ . **It is not leaked supervision:**  $B_1$  is the graph, which the model observes;  $B_2$  is the answer, which is withheld.

7.2 The exactness violation is a calibrated damage measure

Within each topology, nine exactness weights produce learned maps of varying quality. The endpoint is the correlation between a map's exactness violation and its held-out error.

| quantity                             | value            |
|--------------------------------------|------------------|
| mean within-topology correlation     | +0.854           |
| 95% interval                         | [+0.831, +0.877] |
| topologies with positive correlation | 29 / 29          |

Because the sweep happens *inside* a topology, this is genuine within-condition prediction rather than separation between a regularised and an unregularised group.

7.3 The cone harms and RTD is inert

Both are confirmatory. The protocol predicted the cone would fail to improve; it does worse, with an adjusted interval entirely above zero. RTD's unadjusted interval sits marginally above zero, its adjusted interval contains zero, and its sign test is 0.458.

§5.2 gives the mechanism for the cone here: the truth violates the term. §5.1 gives a different mechanism for the same object in the selection setting.

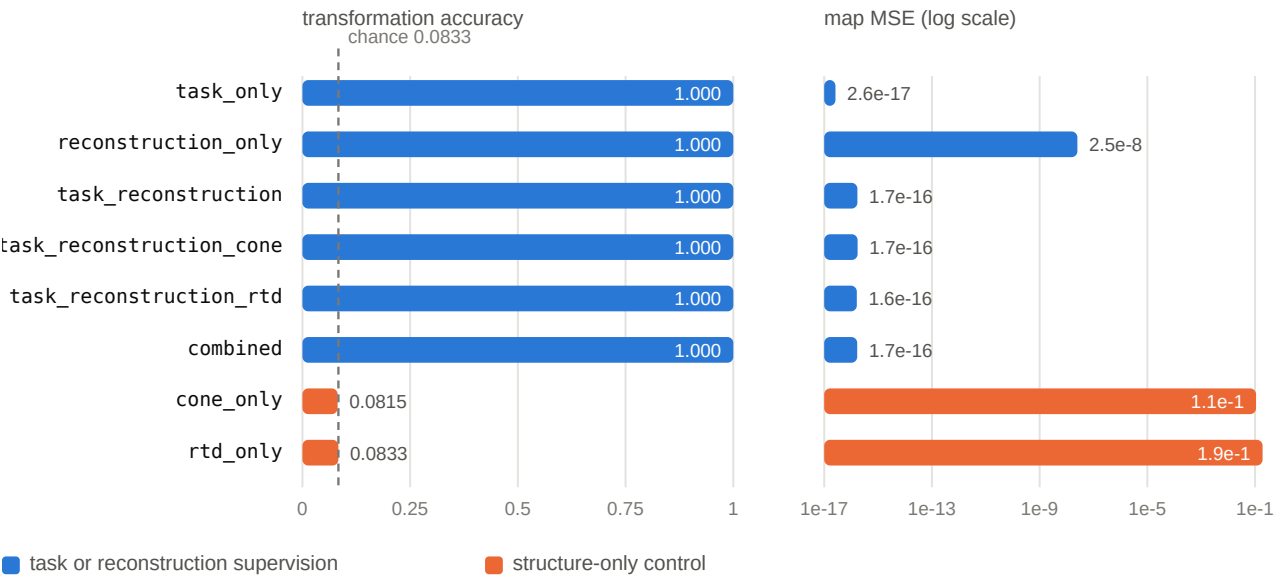


### 7.4 The selection setting, and why its nulls are weak

On the annulus, every objective containing task or reconstruction supervision recovers the planted map exactly — transformation and cell-face accuracy 1.000 on all five seeds, map MSE at 1e-16, engineering recovery gate passed **10 of 10** applicable runs.

#### Exact recovery by training objective

Mean over five seeds. Structure-only objectives sit at chance.



**Figure 2. Recovery by objective in the selection setting.** Mean over five seeds. Any objective carrying task or reconstruction supervision saturates; the two structure-only controls sit at chance with map errors fifteen orders of magnitude larger. Generated from `results/summaries/identifiable-campaign-summary.json`.

All 21 declared continuous contrast intervals contain zero, so adding a cone or RTD term changed nothing. **That null is weak evidence:** an analytic marker decoder also reaches 1.000, so the ceiling was attainable without learning and no candidate could improve on a control already at the maximum.

The informative part is the two structure-only controls. `cone_only` reaches transformation accuracy 0.0815 and `rtd_only` 0.0833 against a 0.0833 chance baseline — **while producing acyclic cones in 6,000 of 6,000 evaluated examples each.** That is §5.1 made visible: the objective is fully satisfied and identification never leaves chance.

### 7.5 Controls: is it exactness, or just regularisation?

Exploratory, ten topologies, at 16 training pairs against plain least squares at 1.582:

| penalty                                                      | median held-out | 95% CI vs plain  |
|--------------------------------------------------------------|-----------------|------------------|
| ridge, best of four weights                                  | 0.631           | [−0.451, −0.174] |
| random subspace, rank matched to <code>B1<sup>T</sup></code> | 2.256           | [−0.103, +0.601] |
| exactness                                                    | 0.002           | [−2.826, −1.134] |

Generic shrinkage buys about 2.5×. **The same quantity of constraint in a random subspace buys nothing.** Exactness buys about 800×. The gain is that specific subspace, not regularisation and not the amount of constraint.

A further exploratory check with a nonlinear link (target `tanh(circulation)`, model `tanh(Wx)`) reduces the gain from roughly 2500× to roughly 13×, interval  $[-1.740, -0.808]$ . Exactness is not solely an artefact of linear least squares, though §9 notes the linear mechanism is understood.

## 7.6 Routing: a compute claim, not an accuracy one

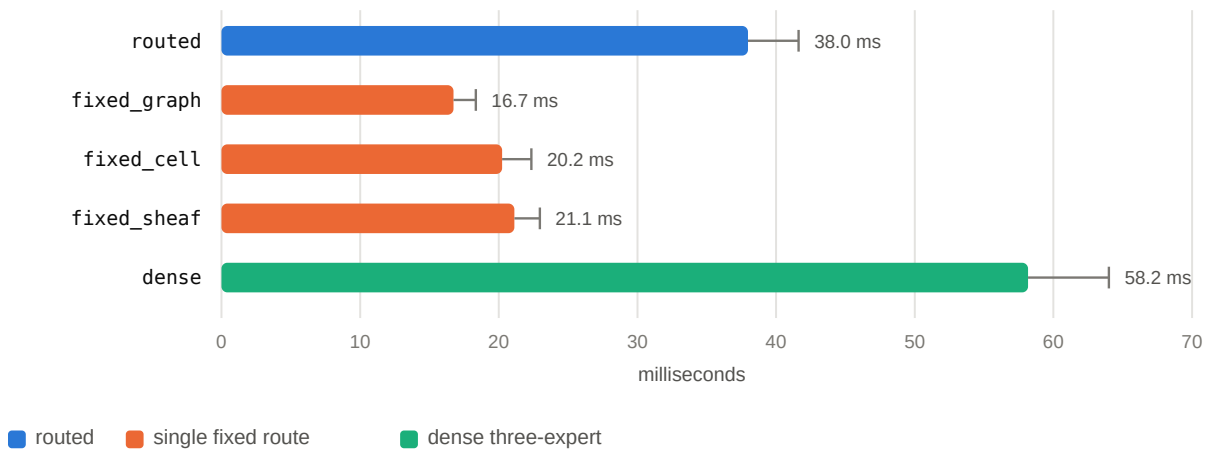
Selecting a view by measured conversion defect was preregistered and is **not supported**: interval  $[-0.111, +0.382]$  contains zero, with the threshold chosen on an even-indexed split and applied unchanged to an odd-indexed evaluation split. A clearly post hoc comparison against the better fixed strategy also contains zero.

The diagnosis is that the setting offered nothing to win. The router picks the better view on **25 of 28** trials, yet a *perfect oracle* buys only **1.36×** over always using the cell view (median 2.022 against 2.741). An 89%-accurate selector cannot beat a fixed choice against that ceiling.

A separate routing result, from a different experiment, does stand — and it is about compute rather than accuracy.

### Trained routing inference latency on GB10

Median over 100 timed iterations, averaged across five seeds; whisker marks mean p95.  
Batch 64, bfloat16. Paths were timed in this order inside one process.



**Figure 3. Trained routing inference latency on GB10.** Median over 100 timed iterations averaged across five seeds; whiskers mark mean p95. Batch 64, bfloat16. All paths were timed in the plotted order inside one process, so residual thermal or allocator drift is confounded with path order. Generated from `results/summaries/compute-campaign.json`.

Routed inference is  $1.532 \pm 0.035\times$  faster than dense three-expert evaluation and  $2.269 \pm 0.043\times$  slower than the fastest single fixed route, with lower peak allocated memory than dense in every seed. The defensible statement is that routing saves compute against dense evaluation, not that it saves compute overall and not that a measured defect selects the view.

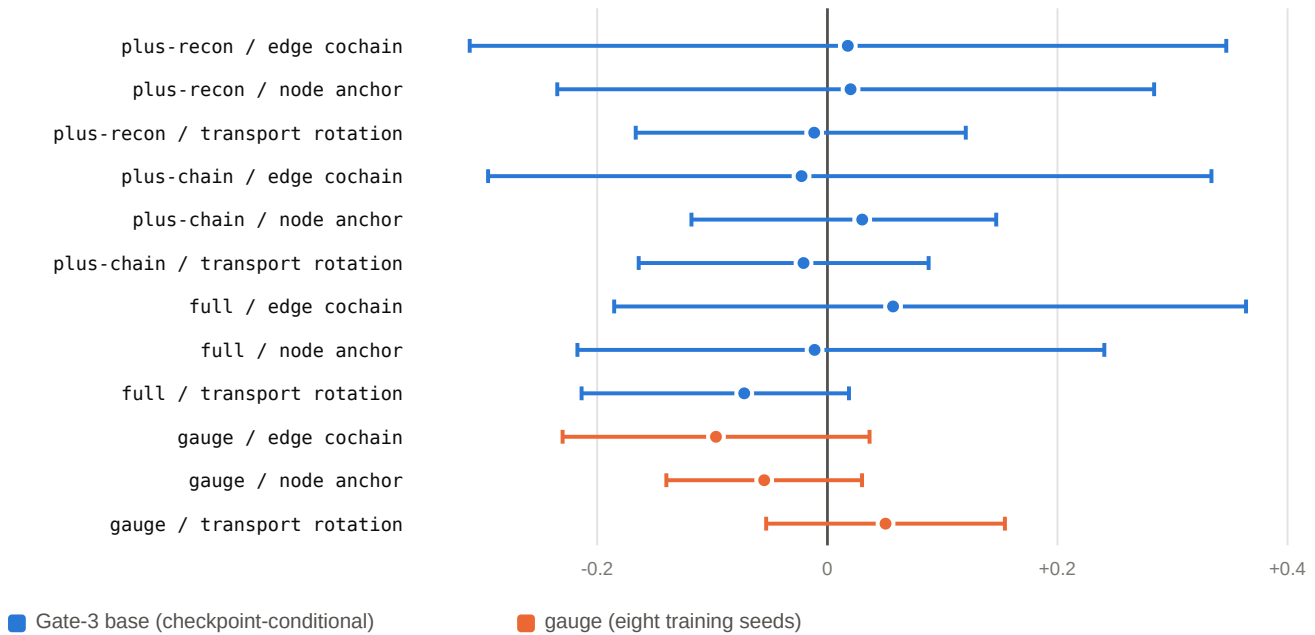
An earlier five-seed campaign found a hard-minus-best-fixed accuracy margin of +0.1098, 95% interval  $[+0.0953, +0.1243]$ . That result used privileged latent-regime distillation and had target structured views available at inference, and a disclosed pre-freeze procedural deviation makes it protocol-aligned rather than pristine preregistration. It is a different experiment from the defect-based routing tested here and the two must not be conflated.

## 7.7 Corruption diagnostics

### Every corruption contrast interval contains zero

Candidate minus baseline on the adjusted partial-Spearman endpoint, with 95% intervals.

Gate-3 rows are conditional on a fixed checkpoint pair; gauge rows are across eight training seeds.



**Figure 4. Every corruption contrast interval contains zero.** Nine Gate-3 base contrasts use a paired complete-block bootstrap conditional on a fixed checkpoint pair; three gauge contrasts use a Student-t interval with  $df = 7$  across eight training seeds. No multiplicity adjustment. Generated from `results/gate3/paired_comparison_final.json` and `results/summaries/gauge-corruption-campaign.json`.

These compare clean and corrupted **fixed-expert embeddings**. They never invoke a translator or learned map and therefore cannot support or refute any conversion claim. All twelve intervals contain zero.

## 8. Discussion

**The choice of homological object decides the outcome.** Three objects, one protocol, three different answers: exactness improves, the cone harms, RTD is inert. Reporting "homological structure does or does not help" without naming the object would have been meaningless.

**Both failures are predictable in advance.** §5's two conditions cost seconds and would have redirected this project years of effort earlier. We regard the screening criterion as the most transferable thing here.

**Acyclicity is not correctness.** A mapping cone certifies that a decoded map is invertible. In a hypothesis class of invertible maps that certificate is constant, and a model trained on it alone satisfies it perfectly while learning nothing.

**The cone bundles what the kernel and cokernel separate.**  $\dim H_n(\text{cone}) = \dim \text{coker } H_n + \dim \ker H_{(n-1)}$  — the cone sums a directional pair and loses the distinction between *destroyed* and *unreachable*. Where a measurement is wanted, the unbundled pair is strictly more informative.

**Exactness works because it is a correct constraint that is free.** The mechanism is not mysterious:  $W B 1^\top = 0$  forces each row of  $W$  into the cycle space, cutting effective dimension from  $F \times E$  to  $F \times F$ . A correct linear constraint helping a linear problem is expected. What is notable is that exactness *is* the correct constraint and is derivable from the input.

## 9. Limitations

---

- **One synthetic family.** 29 topologies, one training-set size, mostly linear conversions. Nothing transfers automatically to real data.
- **The mechanism is understood, not surprising.** See above. The claim is about which constraint is correct and available, not about an unexpected effect.
- **Nonlinearity is only spot-checked.** The `tanh` result is exploratory, at one link and one weight.
- **The selection setting saturates.** Six of eight objectives reach exactly 1.000, so its structural nulls cannot detect improvement.
- **Five seeds in the annulus campaign,** where the exact two-sided sign test floor is  $p = 0.0625$  and can never be decisive.
- **Routing is not established either way.** It failed in a setting with a  $1.36\times$  oracle ceiling; that is a statement about the habitat as much as the method.
- **Privileged supervision** in the historical routing campaign, and **target-view translators** in the historical conversion modules, which consumed target structure and are not conversions.
- **Timing caveats.** All paths timed in fixed order inside one process; raw per-iteration timings not retained; identifiable runner reports p90 and routing runner p95, never pooled.
- **No multiplicity control** outside the three primary conversion contrasts.
- **Artifact boundaries.** The 8.8 GB `artifacts/` tree is untracked; tracked evidence is the curated `results/` bundle.

## 10. Claim boundary

---

This work establishes, on one synthetic family under a frozen protocol, that an input-derivable exactness constraint improves a learned conversion and that its violation calibrates the damage. It establishes that a mapping-cone term harms and an RTD term is inert in the same setting, each for a stated reason.

It does **not** establish:

- any benefit from cone or RTD objectives — the evidence runs the other way;
- that a measured defect can select a representation;
- conversion quality on real or out-of-distribution data;
- general equivalence between graphs, cellular complexes, and sheaves;
- a learned quasi-isomorphism — the verified identity is the chain-map law to a fixed  $1e-5$  tolerance;
- any Langlands, eigensheaf, Fourier–Mukai, or category-theoretic result. Imposing a chain-map constraint by construction is not a categorical claim in the sense of [9] or [10]: we fix one finite hypothesis class by hand in §4.1 and learn inside a fixed nullspace in §4.2.

# 11. Code and data availability

---

**Availability status.** The project repository is private and is licensed proprietarily (Copyright © 2026 Sean Mahdavian, all rights reserved). Code and data are therefore **not publicly available at the time of writing**, and the mechanism by which a reader or reviewer may obtain them is an author decision that must be settled before submission — many venues require either an open release or an explicit, actionable access statement. This section describes what exists and how it is organised, not what is currently downloadable.

All code, configurations, and compact evidence are in that repository. The tracked bundle under `results/` contains 49 files with a `MANIFEST.json` recording path, byte count, SHA-256, generating commit, and generating command for each.

| location                                                   | contents                                                                        |
|------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>results/campaigns/</code>                            | the frozen conversion campaign record                                           |
| <code>results/summaries/</code>                            | strict compact summaries for the annulus, gauge, compute, and routing campaigns |
| <code>results/gate3/</code> , <code>results/gate3g/</code> | gate decisions and per-batch corruption-report derivatives                      |
| <code>results/benchmarks/</code>                           | ten identifiable and five routing trained benchmark records                     |

Corruption reports are exported as per-batch derivatives: the `per_example` array is dropped and the `per_batch` array — the unit of analysis — retained. This is lossless for every published statistic, verified by recomputing the adjusted partial Spearman from retained rows and matching to 1e-15.

Synthetic generators are deterministic given the committed configs and seeds; no external dataset is required.

# 12. Reproducibility

---

```
python scripts/run_conversion_campaign.py \
  --output results/campaigns/conversion-campaign-v1.json
python scripts/summarize_gauge_corruption_campaign.py \
  --output results/summaries/gauge-corruption-campaign.json
python scripts/summarize_compute_campaign.py \
  --output results/summaries/compute-campaign.json
python scripts/export_publication_evidence.py --output-root results
python scripts/export_publication_evidence.py --verify-only
```

Each summarizer revalidates provenance before aggregating and fails closed on any hash, seed, pairing, schema, or receipt mismatch. The exporter works from an explicit allowlist and refuses checkpoints, prediction dumps, histories, logs, and caches. Its manifest carries no timestamp, so re-exporting unchanged evidence reproduces the bundle byte for byte.

The paper PDF is rendered with `python scripts/render_paper.py`; figures are regenerated from tracked evidence with `python scripts/render_figures.py`.

# 13. Declarations

---

**Author contributions.** Sean Mahdavian designed and implemented the system, ran the campaigns, and wrote the manuscript.

**Affiliation.** Phillips Exeter Academy.

**ORCID.** [0009-0000-8432-7825](https://orcid.org/0009-0000-8432-7825).

**Funding.** This work received no external funding.

**Competing interests.** The author declares no competing interests.

**License.** The software, source code, documentation, and evidence described here are released under a proprietary license, Copyright © 2026 Sean Mahdavian, all rights reserved. The terms are in the repository's `LICENSE`. **This is not an open-source or open-data release**, and §11 should be read accordingly.

**Ethics.** Every experiment reported here uses synthetic data generated by committed deterministic code. No human subjects, personal data, or personally identifiable information are involved, and we see no dual-use concern. Compute is small: the annulus campaign is 25.8 minutes of GPU time and the conversion campaign runs on CPU.

A molecular transfer experiment on the public OGBG-MOLHIV benchmark was run earlier in this project and is **deliberately not reported here**, because it answers a different question from the one this paper asks. Its result — the ring-lift route losing to the graph route, with later variants contaminated by repeated inspection of the official test split — is recorded in the project's claims ledger as C6.

**Reporting.** Null and negative results are reported in the body rather than an appendix, and every number in every table is traceable to a tracked machine-readable file under `results/`.

## References

---

1. Barannikov, Trofimov, Balabin, and Burnaev. [Representation Topology Divergence: A Method for Comparing Neural Network Representations](#). ICML, 2022.
2. Trofimov, Cherniavskii, Tulchinskii, Balabin, Burnaev, and Barannikov. [Learning Topology-Preserving Data Representations](#). ICLR, 2023.
3. Wang and Hu. [Symmetric Divergence and Normalized Similarity: A Unified Topological Framework for Representation Analysis](#). TMLR, 2026. arXiv:2606.06342.
4. Beshkov. [A Quotient Homology Theory of Representation in Neural Networks](#). TMLR, 2026.
5. Battiloro, Spinelli, Telyatnikov, Bronstein, Scardapane, and Di Lorenzo. [From Latent Graph to Latent Topology Inference: Differentiable Cell Complex Module](#). ICLR, 2024.
6. Franco, Duarte, Nikitin, Ponti, Mesquita, and Souza. [Differentiable Lifting for Topological Neural Networks](#). NeurIPS 2025 Workshop on Non-Euclidean Foundation Models and Geometric Learning. Workshop poster, not a main-conference paper.
7. Bodnar, Di Giovanni, Chamberlain, Liò, and Bronstein. [Neural Sheaf Diffusion: A Topological Perspective on Heterophily and Oversmoothing in GNNs](#). NeurIPS, 2022.
8. Gebhart, Hansen, and Schrater. [Knowledge Sheaves: A Sheaf-Theoretic Framework for Knowledge Graph Embedding](#). AISTATS, 2023.
9. Gavranović, Lessard, Dudzik, von Glehn, Madeira Araújo, and Veličković. [Position: Categorical Deep Learning is an Algebraic Theory of All Architectures](#). ICML, 2024.
10. Gavranović. [Learning Functors using Gradient Descent](#). EPTCS 323, 2020.
11. Wang, Yu, Dunlap, Ma, Wang, Mirhoseini, Darrell, and Gonzalez. [Deep Mixture of Experts via Shallow Embedding](#). UAI, 2020.

12. H. Wang, Jiang, You, Han, Liu, Srinivasa, Kompella, and Z. Wang. [Graph Mixture of Experts: Learning on Large-Scale Graphs with Explicit Diversity Modeling](#). NeurIPS, 2023.
13. S. Wu, Cao, Ribeiro, Zou, and Leskovec. [GraphMETRO: Mitigating Complex Graph Distribution Shifts via Mixture of Aligned Experts](#). NeurIPS, 2024.
14. Z. Wu, Cai, Zhang, Lu, Chen, Zhuang, and Wang. [Where Graph Meets Heterogeneity: Multi-View Collaborative Graph Experts](#). NeurIPS, 2025.